Part 4: Model & Interpret

For Part 4, I moved from exploratory analysis into predictive modeling in order to test my selected project question:

Can we predict whether a customer will make a repeat purchase based on their first order behavior, payment details, product information, and delivery experience?

According to the project instructions, this stage requires developing hypotheses, testing them using appropriate models, comparing model performance, including at least one visualization to illustrate model performance, and interpreting the results while reflecting on what worked well and what could be improved. In revisiting this stage, I made sure to align the modeling approach more closely with the methods discussed in class by using **multivariate linear regression on numerical features** and comparing two related regression specifications.

My hypothesis for this stage was that customers with stronger first-order spending and better delivery experience would be more likely to make repeat purchases. This hypothesis followed directly from the original project question and from the patterns identified in Part 3. The EDA showed that repeat purchase behavior was highly imbalanced and that variables related to first-order spending and delivery appeared potentially relevant. Based on that, I tested whether a set of first-order numerical variables could explain variation in the repeat-purchase target.

To prepare the data for modeling, I used the final prepared dataset from Part 2 and restricted the modeling input to numerical variables in order to stay within the scope of the regression-based methods emphasized in class. The dependent variable was repeat_purchase_target, coded as a numerical 0/1 outcome. The predictors used came from first-order behavior and included payment value, installments, item count, total price, freight cost, delivery duration, and the delivered-late indicator. I removed rows with missing values in the selected fields and then split the data into training and test sets. After dropping missing values, the modeling dataset contained **93,249 observations**, with **74,599** in the training set and **18,650** in the test set.

model_comparison

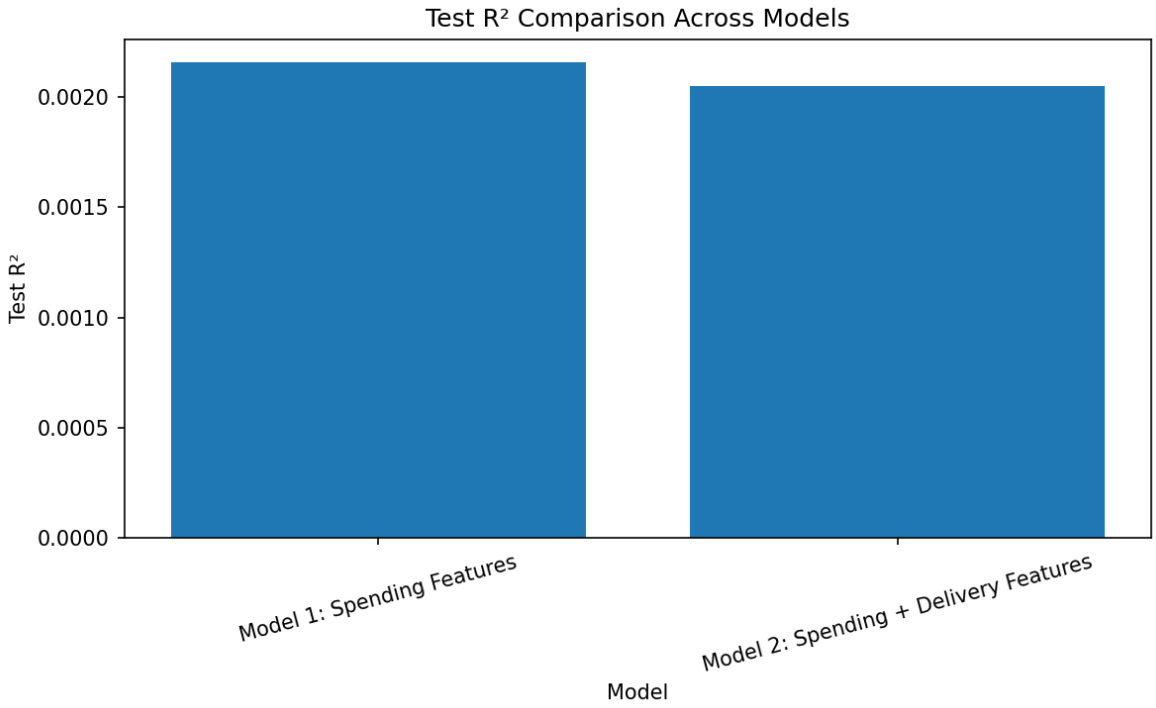
Model	Train_R2	Train_Adjusted_R2	Test_R2
Model 1: Spending Features	0.002117251564587310	0.002050363066441690	0.002156302084950430 0
Model 2: Spending + Delivery Features	0.002240832098756700 0	0.002147197287917350	0.002048268391525610 0

I compared two multivariate OLS regression models. **Model 1** used only spending-related variables: first-order payment value, first-order installments, first-order number of items, first-order total price, and first-order total freight. **Model 2** used those same variables and then added delivery-related variables: delivery duration and

delivered-late status. This comparison was designed to test whether adding delivery features improved the explanatory power of the model beyond the spending variables alone. This setup is consistent with the class emphasis on comparing model specifications and interpreting whether additional predictors improve fit.

The results showed that both models had **very low explanatory power**. Model 1 had a train R^2 of approximately **0.002117**, an adjusted R^2 of **0.002050**, and a test R^2 of **0.002156**. Model 2 had a train R^2 of approximately **0.002241**, an adjusted R^2 of **0.002147**, and a test R^2 of **0.002048**. Although Model 2 had a slightly higher train R^2, Model 1 performed slightly better on the test set. Based on test R^2, **Model 1: Spending Features** was the better model.

r2_comparison.png

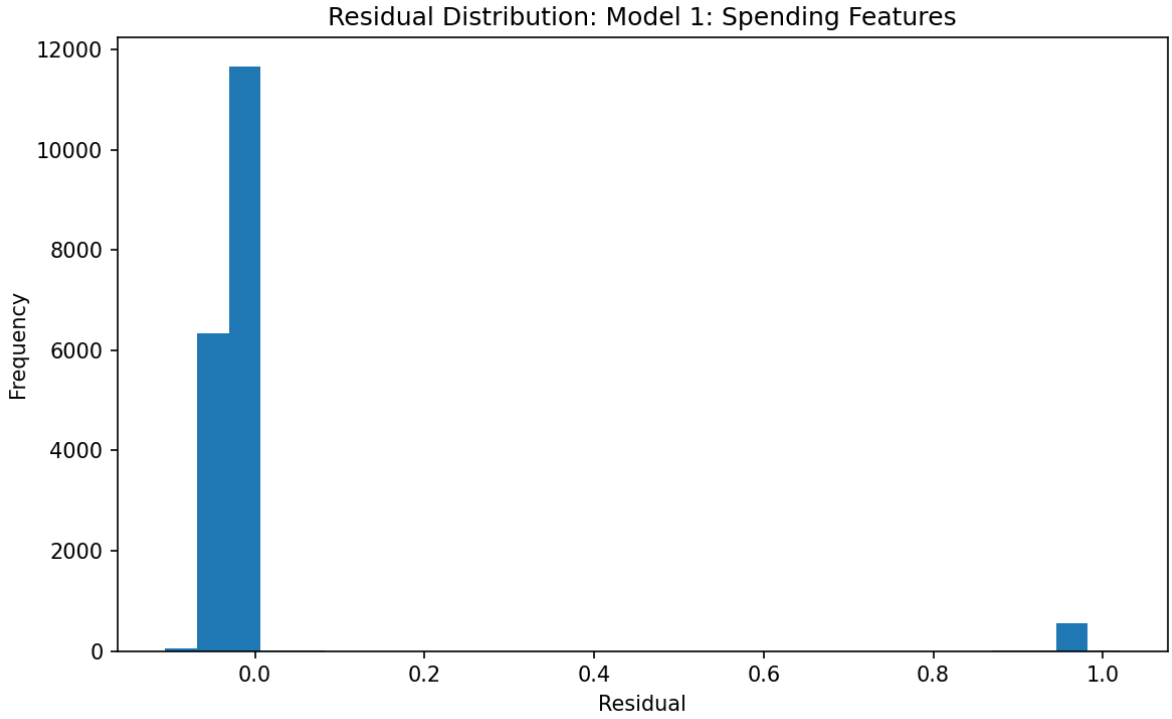


The R^2 comparison chart makes the main result clear: neither model explains much of the variation in repeat purchase behavior, and adding delivery variables does not meaningfully improve performance. This is an important result because it suggests that repeat purchase is not well explained by these first-order numerical variables alone. In other words, while some of the variables may have measurable relationships with the target, the overall prediction problem remains weak from a regression standpoint.

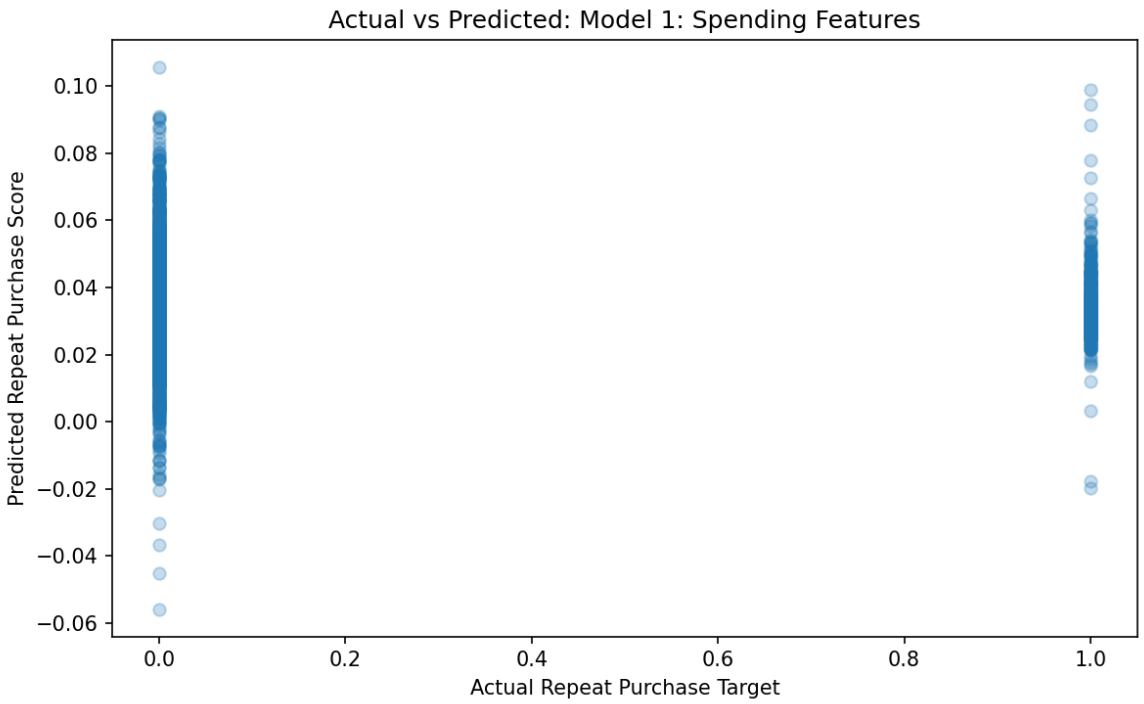
The coefficient estimates also help clarify the results. In both models, several spending-related variables were statistically significant, including first-order payment value, installments, item count, total price, and freight cost. In Model 2, the delivered_late variable was statistically significant, while delivery_duration_days was not statistically significant. However, even with statistically significant

coefficients, the R^2 values remained extremely low. This means that statistical significance should not be confused with strong predictive or explanatory power. The variables may have detectable relationships with repeat purchase, but they explain only a very small amount of the overall variation in customer behavior. This interpretation closely follows the lecture emphasis on reading regression results through coefficients, p-values, and R^2 rather than relying on one number alone.

residual_histogram.png



The residual histogram shows that most residuals are tightly clustered near zero, but the distribution still reflects the limitations of using a linear regression framework on a binary target. This is one reason the fit remains weak. The model does produce numerical predictions, but the relationship between the selected first-order features and repeat purchase is not strong enough to generate meaningful explanatory power at the aggregate level.



The actual-versus-predicted plot further supports this conclusion. The predicted values occupy a narrow range and do not separate the two classes strongly. This means the model is not distinguishing repeat purchasers from non-repeat purchasers in a powerful way. That result is consistent with the low R^2 values and reinforces the conclusion that repeat purchase is influenced by many factors that are not captured in the available first-order numeric features.

Taken together, the model comparison suggests that **Model 1, using only spending-related features, performed slightly better than Model 2 on the test data**, even though both models were weak overall. This means that delivery-related variables did not add meaningful predictive value in this regression setup. The hypothesis was only partially supported. Some spending features and the delivered-late indicator showed statistically significant relationships with the target, but the overall model fit remained extremely low, so the practical explanatory value of the models is limited.

What worked well in this stage was that the project followed the full course workflow: structured multi-table data, SQL preparation, feature engineering, EDA, and then numerical modeling in Python. The regression comparison was useful because it showed that adding more variables does not automatically improve out-of-sample performance. What did not work as well was the overall fit of the models. With more time, I would consider trying different numerical feature constructions, exploring additional variables, and testing whether other course-aligned specifications or transformations could better capture customer repeat behavior. I would also pay closer attention to multicollinearity, since the OLS summaries reported a relatively high condition number, suggesting some overlap among predictors.

Overall, this version of Part 4 satisfies the project requirement to develop hypotheses, compare models, include visualizations, and interpret the results. The main takeaway is that repeat purchase is difficult to explain using only the available first-order numerical features, and that a simpler spending-only model performed slightly better than the expanded spending-plus-delivery model on the test data.

AI Disclosure

I used ChatGPT to help organize the modeling workflow, and refine the written interpretation of the results.

Part 3: Explore Data

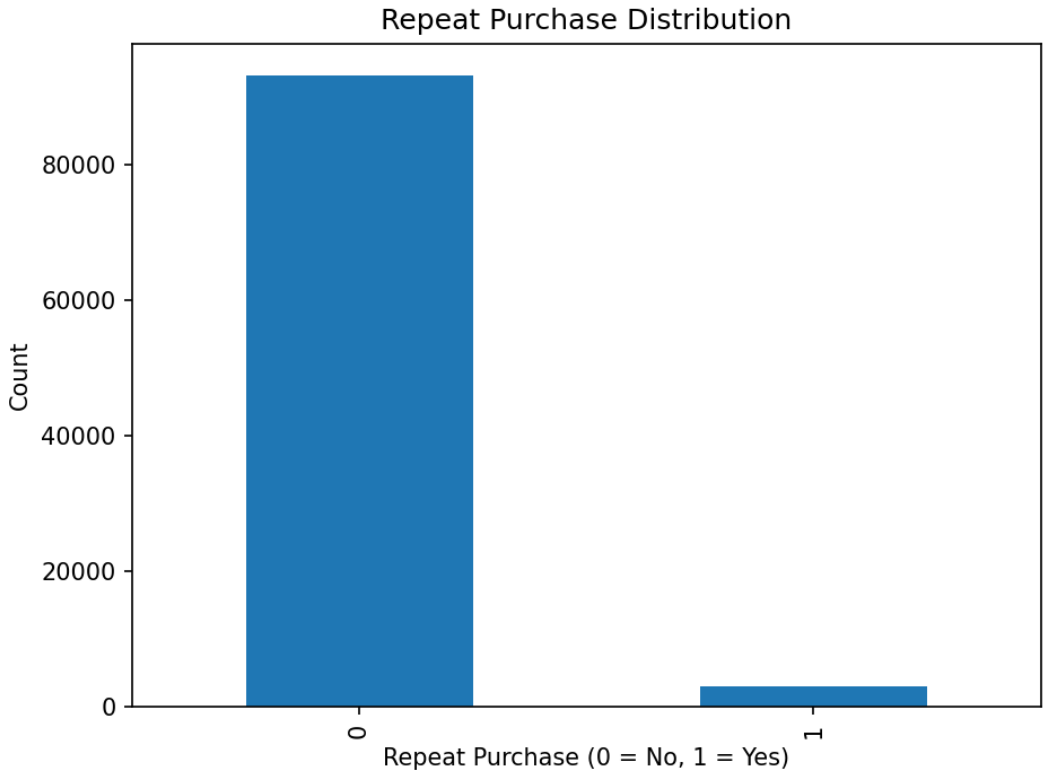
For Part 3, I conducted an Exploratory Data Analysis on the final prepared dataset created in Part 2 in order to identify patterns, data quality issues, and relationships relevant to my selected prediction question:

Can we predict whether a customer will make a repeat purchase based on their first order behavior, payment details, product information, and delivery experience?

According to the project instructions, Part 3 requires an EDA that uncovers patterns, trends, and data quality issues with a particular emphasis on the target variable. The instructions also say that the findings should be interpreted in relation to the problem statement, not just described, and that the submission should include at least two clear and well-labeled visualizations along with a written analysis and supporting code or queries. I structured this stage around those requirements by analyzing the final customer-level dataset prepared in Part 2 and focusing on how the engineered first-order features relate to repeat purchase behavior.

The final dataset contained one row per customer and included the target variable repeat_purchase_target along with first-order payment, order, product, freight, and delivery features. Before making visualizations, I first examined the structure of the dataset by checking the shape, column names, data types, and missing values. This step was important because Part 3 is not only about pattern-finding, but also about noticing data quality issues that may affect interpretation or later modeling. The dataset contained several missing values in a few engineered variables, including small numbers of missing values in payment-related fields and larger numbers of missing values in product-category and delivery-duration fields. These missing values are meaningful because they suggest that some first orders did not have fully available product or delivery information, which may need to be handled carefully during the modeling stage.

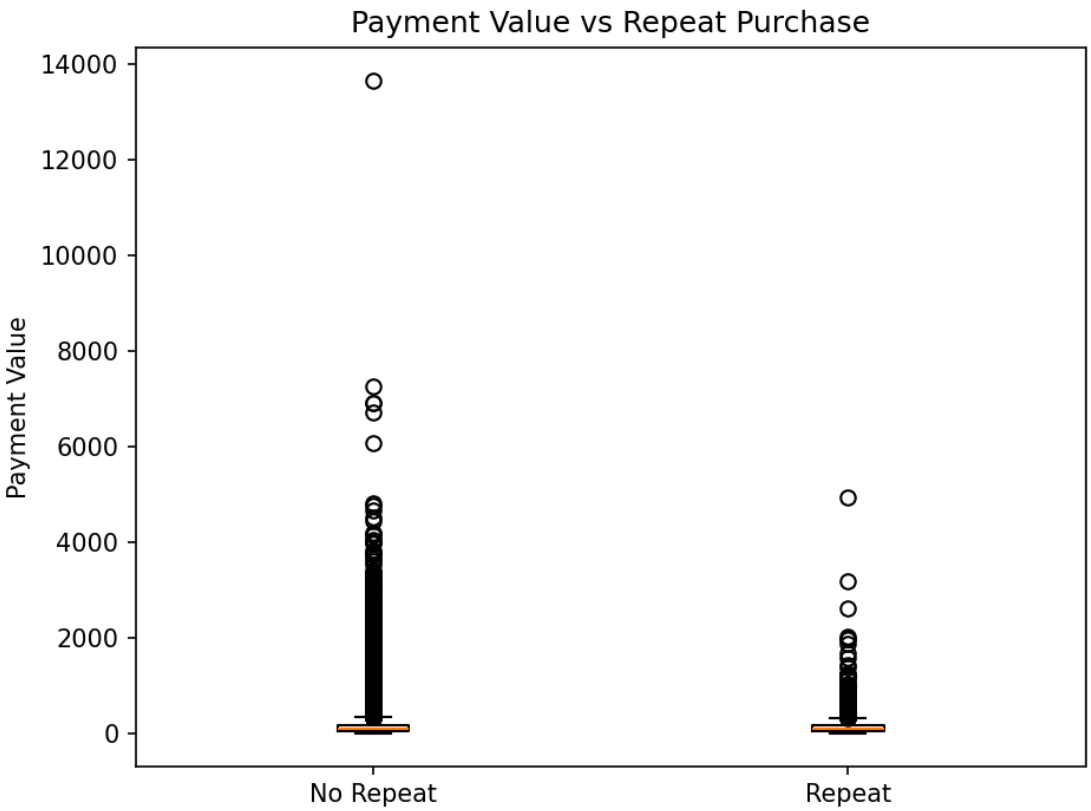
The first major focus of the EDA was the **target variable**. The distribution of repeat_purchase_target showed a strong class imbalance. Most customers in the dataset did not make a repeat purchase, while a much smaller number did. This is an important finding because it tells us that repeat purchasing is relatively uncommon in this sample, and that any later prediction model will need to account for that imbalance. More importantly, this directly relates to the project question, since the goal is to understand which first-order characteristics may help distinguish the smaller repeat-purchase group from the much larger non-repeat group.



[Figure 1 — repeat_purchase_distribution.png]

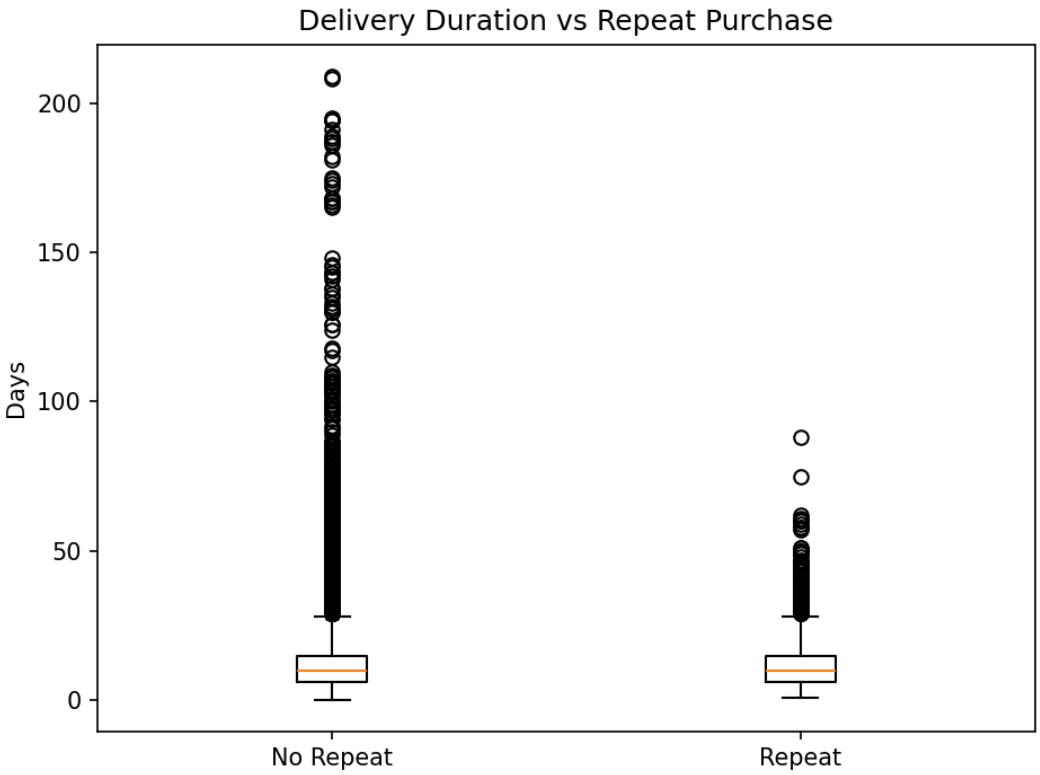
The first visualization, a bar chart of the target variable, clearly shows that non-repeat customers dominate the dataset. This helps communicate the structure of the prediction task and confirms that the target is not evenly distributed. That matters because patterns associated with repeat purchase may be harder to identify if they are overwhelmed by the majority class. This makes the target distribution itself one of the most important early findings in the exploratory analysis.

The second major focus was **first-order payment behavior**. To explore this, I compared first_order_payment_value across repeat and non-repeat customers using a boxplot. The purpose of this visualization was to see whether customers who eventually returned tended to have a different spending pattern on their first order. The plot showed substantial variability and many outliers in both groups, which suggests that first-order payment value is highly skewed. However, the visualization still provides a useful comparison because it shows that spending behavior varies considerably and may contain predictive signal, even if it is noisy. The presence of extreme values also suggests that this variable may need careful treatment in the modeling stage, such as scaling, transformation, or outlier-aware interpretation.



[Figure 2 — payment_value_vs_repeat_purchase.png]

The third major focus was **delivery experience**, since that was explicitly included in my original proposal. I examined delivery_duration_days by repeat-purchase group using a second boxplot. This plot suggested that delivery experience may differ between repeat and non-repeat customers, especially because the non-repeat group appeared to contain more extreme long-duration outliers. While the central tendency of the two groups may not be drastically different at first glance, the outlier pattern is still meaningful. A very long delivery experience could negatively affect customer satisfaction and reduce the likelihood of future purchases. This means that even if delivery duration is not the only important variable, it is still a reasonable feature to keep and investigate further in later stages of the project.



[Figure 3 — delivery_duration_vs_repeat_purchase.png]

Overall, the EDA revealed three important insights that directly connect back to the problem statement. First, repeat purchase behavior is highly imbalanced, meaning repeat customers form a relatively small subgroup in the data. Second, first-order payment value shows a wide spread and many outliers, suggesting that customer spending behavior may contain predictive information but also requires careful interpretation. Third, delivery experience appears relevant, especially because unusually long delivery times are more visible among non-repeat customers. In addition, the missing-value analysis showed that some engineered variables are incomplete, particularly delivery duration and product category, which is important to keep in mind before modeling.

Taken together, these findings suggest that the features created in Part 2 are meaningful for the repeat-purchase question and that the final prepared dataset is suitable for the next modeling stage. The EDA did not just describe the dataset; it highlighted target imbalance, data quality issues, and plausible feature-target relationships that are directly relevant to the goal of predicting repeat purchase behavior. This is exactly the type of exploratory analysis the project instructions call for before moving into model development.

AI Disclosure

I used ChatGPT to help organize the EDA workflow, refine the wording of the written analysis, and suggest clear ways to interpret the visualizations in relation to the project question.

Part 2: Prepare Data

For Part 2, I cleaned, organized, and combined the Olist e-commerce data in order to prepare a final dataset for my selected prediction question:

Can we predict whether a customer will make a repeat purchase based on their first order behavior, payment details, product information, and delivery experience?

This question came directly from my Part 1 proposal, so the goal of this stage was to make sure the data structure matched that problem correctly. According to the project instructions, Part 2 requires cleaning and normalizing the data, combining tables as needed, ideally setting up a database and using SQL queries, joining and aggregating information where appropriate, and creating any necessary new features with an explanation of why they were added. I followed that structure directly by setting up a cloud-based PostgreSQL database through Neon and connecting to it through pgAdmin, then using SQL to create and populate the tables, inspect the data, and build a final analysis-ready dataset.

I first imported the five core tables that were most relevant to my project question: customers, orders, order_items, order_payments, and products. I chose these tables because they directly matched the components of my proposal. The customers and orders tables were needed to identify customers and their purchase history, order_payments was needed for payment behavior, order_items and products were needed for first-order product and pricing information, and the orders table also contained the delivery dates needed to measure delivery experience. This approach followed the proposal closely instead of adding unrelated tables or features. It also matched the Part 2 instruction to combine tables as needed and prepare the data using SQL.

After importing the tables, I ran basic data checks before creating any features. I checked row counts across the five tables, previewed sample rows, checked key identifier columns for missing values, and verified that the timestamp fields in the orders table had loaded correctly. This step was important because Part 2 is not just about importing data. It is specifically about cleaning and preparing the data in a thoughtful way before analysis. The row counts and previews showed that the relational structure of the data had loaded properly, and the timestamp checks confirmed that the purchase and delivery date columns could be used for time-based features later on.

The most important design choice in this stage was deciding the correct **unit of analysis**. Since my question is about whether a **customer** makes a repeat purchase, the final dataset could not be one row per order. Instead, it had to be one row per customer, with features taken from that customer's **first order** and a target variable indicating whether that customer purchased again later. This was necessary in order

to keep the prepared dataset aligned with the proposal and to avoid mixing order-level and customer-level logic. To do this correctly, I used customer_unique_id instead of customer_id, because customer_unique_id represents the true customer across purchases, while customer_id is order-specific in this dataset. Using customer_unique_id allowed me to count repeat purchases accurately. Based on this logic, I created a repeat purchase target where customers with more than one distinct order were labeled 1, and customers with exactly one order were labeled 0. This directly operationalized the prediction question from the proposal.

Next, I created a view containing only the **first order** for each customer. I used a window function with ROW_NUMBER() partitioned by customer_unique_id and ordered by order_purchase_timestamp so that I could isolate the earliest order for every customer. This was an important step because my proposal explicitly states that the model should use **first order behavior**. Once I had that first-order view, I began engineering features that matched the question.

For **payment details**, I joined first orders to the order_payments table and created features such as total payment value for the first order, number of installments, and payment type. These variables capture how the first order was paid for and may help explain later customer behavior.

For **product information and first-order behavior**, I joined first orders to order_items and products. From that, I created features such as number of items in the first order, total item price, total freight cost, and a product category variable. These features reflect the size and composition of the first purchase, which may be related to whether the customer returns.

For **delivery experience**, I used the order and delivery timestamps from the orders table to create two additional variables: delivery_duration_days and delivered_late. delivery_duration_days measures how long it took from purchase to delivery, while delivered_late indicates whether the delivered date was later than the estimated delivery date. These were added because the proposal explicitly includes delivery experience as one of the predictors of repeat purchase behavior.

Finally, I joined all of these intermediate views together into one final prepared dataset called final_repeat_purchase_dataset. This final dataset contains one row per customer and includes the repeat purchase target along with first-order payment, order, product, freight, and delivery features. The final dataset contains **96,096 rows**, which is reasonable because it is aggregated at the customer_unique_id level rather than the raw row level of the original tables. At this point, the dataset is fully prepared for the next project stage, since it is now organized in a way that directly matches the chosen predictive question and can be used for both exploratory analysis and modeling.

Overall, this Part 2 process followed the project instructions closely. I set up and used my own database, wrote SQL queries to clean and join the tables, created new features tied directly to my research question, and prepared a final dataset whose

structure fits the prediction task in my proposal. The result is a clean customer-level dataset that is ready for Part 3 exploratory analysis and Part 4 modeling.

AI Disclosure

I used ChatGPT to help refine the wording of my Part 2 explanation, organize the SQL workflow, and clarify how to describe the feature engineering and table-joining process

---

Part 1: Acquire & Define

For my final project, I chose the **Brazilian E-Commerce Public Dataset by Olist** from Kaggle. I picked this dataset because it fits the project requirements well and gives me enough structure to do a full data science workflow without making the project unnecessarily messy. The project prompt asks for a dataset with at least two connected tables, enough size and complexity for meaningful analysis, and a problem statement that matches how the tables connect. This dataset works well for that because it is relational, CSV-friendly, and supports multiple realistic prediction tasks. The data sources guide also specifically recommends using CSV files, preferring datasets with multiple tables or linkable IDs, and choosing something that can support a predictive question.

The Kaggle dataset contains about **100,000 orders** from **2016 to 2018** and includes information across multiple connected tables such as customers, orders, order items, payments, products, reviews, and geolocation. Kaggle describes it as real anonymized commercial e-commerce data that lets you view an order from multiple dimensions, including order status, price, payment and freight performance, customer location, product attributes, and reviews. That is a big reason why I chose it. It is not just a single flat file. It gives enough relational depth to do joins properly and build a more thoughtful project.

Data Source

- **Kaggle:** Brazilian E-Commerce Public Dataset by Olist. (https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce/data?select=olist_products_dataset.csv)

Why I chose this dataset

I chose this dataset because it feels like the strongest fit for the assignment. First, it has multiple connected tables, which is something the prompt strongly encourages. Second, it is in CSV format, which makes it easier to import into SQL and Python, and the data sources guide says CSV is the recommended format for this project. Third, it supports several business-style predictive questions that are specific and realistic, instead of forcing a vague or weak topic. I also like that it gives me room to join tables, create features, and compare different possible target variables later on in the

project. That lines up well with the rest of the assignment, especially the later parts involving SQL-based preparation, EDA tied to the target variable, and modeling.

Candidate Problem Statements

Since Part 1 asks for a problem statement and this is still the early stage of the project, I am proposing three candidate questions and plan to finalize one after feedback.

1. **Repeat Purchase Prediction:** Can we predict whether a customer will make a repeat purchase based on their first order behavior, payment details, product information, and delivery experience?
2. **Delivery Delay Prediction:** Can we predict whether an order will be delivered late based on order timing, product characteristics, payment details, and shipping-related information?
3. **Low Review Score Prediction:** Can we predict whether a customer will leave a low review score based on product, payment, and delivery characteristics?

I chose these three because they are all specific, predictive, and clearly tied to the way the tables connect. The data sources guide specifically recommends framing the project around a predictive question in the form of "Can we predict ___ using ___?", so these questions are built with that in mind. They also each make use of multiple linked tables instead of relying on just one isolated file.

Brief Description of the Tables

The main tables I plan to use are the following:

- **Customers table**: contains customer identifiers and location information such as city, state, and zip code prefix.
- **Orders table**: contains order-level information such as order status, purchase timestamp, delivery dates, and estimated delivery date.
- **Order Items table**: contains product-level details within each order, including product ID, seller ID, price, and freight value.
- **Order Payments table**: contains payment information for each order, such as payment type, installments, and payment value.
- **Products table**: contains product metadata and category information.
- **Order Reviews table**: contains review scores and review timestamps, which are especially useful if I go with the customer satisfaction question.

Kaggle's dataset description and related schema references show that the data is organized across multiple datasets for better understanding and that the core e-commerce data includes customers, orders, order items, products, payments, reviews, and geolocation.

Table Relationships

These tables are connected through shared IDs, which is what makes the dataset a good fit for relational analysis:

- customers connects to orders through **customer_id**
- orders connects to order_items through **order_id**
- orders connects to order_payments through **order_id**
- orders connects to order_reviews through **order_id**
- order_items connects to products through **product_id**

These relationships make it possible to combine customer behavior, order details, payment patterns, product information, and review outcomes into one analysis pipeline. That is exactly the kind of multi-table structure the project is looking for.

Why these problem statements make sense for this dataset

I think these candidate questions make sense because they are all realistic and directly connected to what the data actually contains. The repeat purchase question uses customer and order history. The delivery delay question uses timestamps and order logistics. The low review score question uses reviews together with order, product, and delivery information. So none of these feel forced. They all come naturally from the available tables, and each one could lead to a solid classification project with meaningful interpretation later on. Since the prompt says a strong Part 1 submission should have a specific problem statement that aligns with the dataset and clear descriptions of the table relationships, I wanted to choose questions that clearly satisfy that.

Short AI Disclosure Note

I used ChatGPT to help refine the wording of my dataset justification, organize the table descriptions and overall for grammatical errors.

SQL + PYTHON CODE:

```
/*
-- step 1A
SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
UNION ALL
SELECT 'orders', COUNT(*) FROM orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM order_items
UNION ALL
```

```
SELECT 'order_payments', COUNT(*) FROM order_payments
UNION ALL
SELECT 'products', COUNT(*) FROM products;

-- step 1B
SELECT * FROM customers LIMIT 5;
SELECT * FROM orders LIMIT 5;
SELECT * FROM order_items LIMIT 5;
SELECT * FROM order_payments LIMIT 5;
SELECT * FROM products LIMIT 5;

-- Step 1C
SELECT COUNT(*) AS missing_customer_id
FROM customers
WHERE customer_id IS NULL;

SELECT COUNT(*) AS missing_customer_unique_id
FROM customers
WHERE customer_unique_id IS NULL;

SELECT COUNT(*) AS missing_order_id
FROM orders
WHERE order_id IS NULL;

SELECT COUNT(*) AS missing_customer_id_in_orders
FROM orders
WHERE customer_id IS NULL;

SELECT COUNT(*) AS missing_product_id
FROM order_items
WHERE product_id IS NULL;

-- step 1D
SELECT
    MIN(order_purchase_timestamp) AS min_purchase_time,
    MAX(order_purchase_timestamp) AS max_purchase_time,
    MIN(order_delivered_customer_date) AS min_delivered_time,
    MAX(order_delivered_customer_date) AS max_delivered_time
FROM orders;
```

```
-- step 2A
CREATE OR REPLACE VIEW customer_order_counts AS
SELECT
    c.customer_unique_id,
    COUNT(DISTINCT o.order_id) AS total_orders,
    CASE
        WHEN COUNT(DISTINCT o.order_id) > 1 THEN 1
        ELSE 0
    END AS repeat_purchase_target
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_unique_id;

-- step 2B
SELECT * FROM customer_order_counts
LIMIT 10;

-- step 3A
CREATE OR REPLACE VIEW first_orders AS
SELECT *
FROM (
    SELECT
        c.customer_unique_id,
        o.order_id,
        o.customer_id,
        o.order_status,
        o.order_purchase_timestamp,
        o.order_approved_at,
        o.order_delivered_carrier_date,
        o.order_delivered_customer_date,
        o.order_estimated_delivery_date,
        ROW_NUMBER() OVER (
            PARTITION BY c.customer_unique_id
            ORDER BY o.order_purchase_timestamp
        ) AS rn
    FROM customers c
    JOIN orders o
        ON c.customer_id = o.customer_id
) t
```

```
WHERE rn = 1;

-- step 3B
SELECT * FROM first_orders
LIMIT 10;

-- step 4A
CREATE OR REPLACE VIEW first_order_payments AS
SELECT
    fo.customer_unique_id,
    fo.order_id,
    SUM(op.payment_value) AS first_order_payment_value,
    MAX(op.payment_installments) AS first_order_installments,
    MIN(op.payment_type) AS first_order_payment_type
FROM first_orders fo
LEFT JOIN order_payments op
    ON fo.order_id = op.order_id
GROUP BY fo.customer_unique_id, fo.order_id;

-- step 4B
SELECT * FROM first_order_payments
LIMIT 10;

-- step 5A
CREATE OR REPLACE VIEW first_order_items AS
SELECT
    fo.customer_unique_id,
    fo.order_id,
    COUNT(oi.product_id) AS first_order_num_items,
    SUM(oi.price) AS first_order_total_price,
    SUM(oi.freight_value) AS first_order_total_freight,
    MIN(p.product_category_name) AS first_order_main_category
FROM first_orders fo
LEFT JOIN order_items oi
    ON fo.order_id = oi.order_id
LEFT JOIN products p
    ON oi.product_id = p.product_id
GROUP BY fo.customer_unique_id, fo.order_id;

-- step 5B
```

```
SELECT * FROM first_order_items
LIMIT 10;

-- step 6A
CREATE OR REPLACE VIEW first_order_delivery AS
SELECT
    customer_unique_id,
    order_id,
    order_status,
    order_purchase_timestamp,
    order_delivered_customer_date,
    order_estimated_delivery_date,
    CASE
        WHEN order_delivered_customer_date IS NOT NULL
         AND order_estimated_delivery_date IS NOT NULL
        THEN EXTRACT(DAY FROM (order_delivered_customer_date -
order_purchase_timestamp))
        ELSE NULL
    END AS delivery_duration_days,
    CASE
        WHEN order_delivered_customer_date IS NOT NULL
         AND order_estimated_delivery_date IS NOT NULL
         AND order_delivered_customer_date >
order_estimated_delivery_date
        THEN 1
        ELSE 0
    END AS delivered_late
FROM first_orders;

-- step 6B
SELECT * FROM first_order_delivery
LIMIT 10;

-- step 7A
CREATE OR REPLACE VIEW final_repeat_purchase_dataset AS
SELECT
    coc.customer_unique_id,
    coc.repeat_purchase_target,
    fop.first_order_payment_value,
    fop.first_order_installments,
    fop.first_order_payment_type,
```

```
    foi.first_order_num_items,
    foi.first_order_total_price,
    foi.first_order_total_freight,
    foi.first_order_main_category,
    fod.order_status,
    fod.delivery_duration_days,
    fod.delivered_late
FROM customer_order_counts coc
LEFT JOIN first_order_payments fop
    ON coc.customer_unique_id = fop.customer_unique_id
LEFT JOIN first_order_items foi
    ON coc.customer_unique_id = foi.customer_unique_id
LEFT JOIN first_order_delivery fod
    ON coc.customer_unique_id = fod.customer_unique_id;

-- step 7B
SELECT * FROM final_repeat_purchase_dataset
LIMIT 20;

-- step 7C
SELECT COUNT(*) AS final_rows
FROM final_repeat_purchase_dataset;
SELECT * FROM final_repeat_purchase_dataset ;

*/
```

PART 3:

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("final_repeat_purchase_dataset.csv")

# ----------------------
# 1. Basic overview
# ----------------------
print("Shape:", df.shape)
print("\nColumns:\n", df.columns)
```

```
print("\nData types:\n", df.dtypes)

# ----------------------
# 2. Missing values
# ----------------------
print("\nMissing values:\n", df.isnull().sum())

# ----------------------
# 3. Target variable analysis
# ----------------------
print("\nTarget distribution:\n",
df["repeat_purchase_target"].value_counts())

df["repeat_purchase_target"].value_counts().plot(kind="bar")
plt.title("Repeat Purchase Distribution")
plt.xlabel("Repeat Purchase (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("repeat_purchase_distribution.png", dpi=150)
plt.close()

# ----------------------
# 4. Payment value vs target
# ----------------------
plt.boxplot([
    df[df["repeat_purchase_target"] == 0]
["first_order_payment_value"].dropna(),
    df[df["repeat_purchase_target"] == 1]
["first_order_payment_value"].dropna()
])
plt.xticks([1, 2], ["No Repeat", "Repeat"])
plt.title("Payment Value vs Repeat Purchase")
plt.ylabel("Payment Value")
plt.tight_layout()
plt.savefig("payment_value_vs_repeat_purchase.png", dpi=150)
plt.close()
```

```
# ----------------------
# 5. Delivery duration vs target
# ----------------------
plt.boxplot([
    df[df["repeat_purchase_target"] == 0]
["delivery_duration_days"].dropna(),
    df[df["repeat_purchase_target"] == 1]
["delivery_duration_days"].dropna()
])
plt.xticks([1, 2], ["No Repeat", "Repeat"])
plt.title("Delivery Duration vs Repeat Purchase")
plt.ylabel("Days")
plt.tight_layout()
plt.savefig("delivery_duration_vs_repeat_purchase.png", dpi=150)
plt.close()

print("\nSaved plots:")
print("- repeat_purchase_distribution.png")
print("- payment_value_vs_repeat_purchase.png")
print("- delivery_duration_vs_repeat_purchase.png”)
```

PART4:

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import statsmodels.api as sm

from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

# ----------------------
# 1. Load data
# ----------------------
df = pd.read_csv("final_repeat_purchase_dataset.csv")
```

```
print("Original shape:", df.shape)

# -----------------------
# 2. Keep only numerical variables
# -----------------------
cols_needed = [
    "repeat_purchase_target",
    "first_order_payment_value",
    "first_order_installments",
    "first_order_num_items",
    "first_order_total_price",
    "first_order_total_freight",
    "delivery_duration_days",
    "delivered_late"
]

df_model = df[cols_needed].copy()

# Drop missing values
df_model = df_model.dropna()

print("Shape after dropping missing values:", df_model.shape)
print("\nColumns used:\n", df_model.columns)

# -----------------------
# 3. Define target and model feature sets
# -----------------------
y = df_model["repeat_purchase_target"]

model1_features = [
    "first_order_payment_value",
    "first_order_installments",
    "first_order_num_items",
    "first_order_total_price",
    "first_order_total_freight"
]
```

```
model2_features = model1_features + [
    "delivery_duration_days",
    "delivered_late"
]

# -----------------------
# 4. Train-test split
# -----------------------
train_df, test_df = train_test_split(
    df_model,
    test_size=0.2,
    random_state=42,
    stratify=df_model["repeat_purchase_target"]
)

print("\nTrain shape:", train_df.shape)
print("Test shape:", test_df.shape)

# -----------------------
# 5. Fit helper
# -----------------------
def fit_ols_model(train_df, test_df, feature_list, model_name):
    X_train = train_df[feature_list]
    y_train = train_df["repeat_purchase_target"]

    X_test = test_df[feature_list]
    y_test = test_df["repeat_purchase_target"]

    X_train_const = sm.add_constant(X_train)
    X_test_const = sm.add_constant(X_test, has_constant="add")

    model = sm.OLS(y_train, X_train_const).fit()
    preds = model.predict(X_test_const)

    test_r2 = r2_score(y_test, preds)
```

```
    residuals = y_test - preds

    print(f"\n{model_name} Summary:")
    print(model.summary())

    return {
        "name": model_name,
        "model": model,
        "preds": preds,
        "test_r2": test_r2,
        "residuals": residuals
    }

# -----------------------
# 6. Fit both models
# -----------------------
result1 = fit_ols_model(train_df, test_df, model1_features, "Model
1: Spending Features")
result2 = fit_ols_model(train_df, test_df, model2_features, "Model
2: Spending + Delivery Features")

# -----------------------
# 7. Compare models
# -----------------------
comparison_df = pd.DataFrame({
    "Model": [result1["name"], result2["name"]],
    "Train_R2": [result1["model"].rsquared,
result2["model"].rsquared],
    "Train_Adjusted_R2": [result1["model"].rsquared_adj,
result2["model"].rsquared_adj],
    "Test_R2": [result1["test_r2"], result2["test_r2"]]
})

print("\nModel Comparison:")
print(comparison_df)

comparison_df.to_csv("model_comparison.csv", index=False)
```

```
# ----------------------
# 8. Visualization 1: R² comparison
# ----------------------
plt.figure(figsize=(8, 5))
plt.bar(comparison_df["Model"], comparison_df["Test_R2"])
plt.title("Test R² Comparison Across Models")
plt.xlabel("Model")
plt.ylabel("Test R²")
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig("r2_comparison.png", dpi=150)
plt.close()

# ----------------------
# 9. Pick better model based on Test R²
# ----------------------
best_result = result1 if result1["test_r2"] >= result2["test_r2"]
else result2

print("\nBest model based on Test R²:", best_result["name"])

# ----------------------
# 10. Visualization 2: Residual histogram for best model
# ----------------------
plt.figure(figsize=(8, 5))
plt.hist(best_result["residuals"], bins=30)
plt.title(f"Residual Distribution: {best_result['name']}")
plt.xlabel("Residual")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("residual_histogram.png", dpi=150)
plt.close()

# ----------------------
# 11. Visualization 3: Actual vs predicted scatter for best model
# ----------------------
```

```
plt.figure(figsize=(8, 5))
plt.scatter(test_df["repeat_purchase_target"],
best_result["preds"], alpha=0.25)
plt.title(f"Actual vs Predicted: {best_result['name']}")
plt.xlabel("Actual Repeat Purchase Target")
plt.ylabel("Predicted Repeat Purchase Score")
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=150)
plt.close()

# ----------------------
# 12. Save coefficient tables
# ----------------------
coef_table_1 = pd.DataFrame({
    "Variable": result1["model"].params.index,
    "Coefficient": result1["model"].params.values,
    "P_Value": result1["model"].pvalues.values
})
coef_table_1.to_csv("model1_coefficients.csv", index=False)

coef_table_2 = pd.DataFrame({
    "Variable": result2["model"].params.index,
    "Coefficient": result2["model"].params.values,
    "P_Value": result2["model"].pvalues.values
})
coef_table_2.to_csv("model2_coefficients.csv", index=False)

print("\nSaved files:")
print("- model_comparison.csv")
print("- r2_comparison.png")
print("- residual_histogram.png")
print("- actual_vs_predicted.png")
print("- model1_coefficients.csv")
print("- model2_coefficients.csv")
```